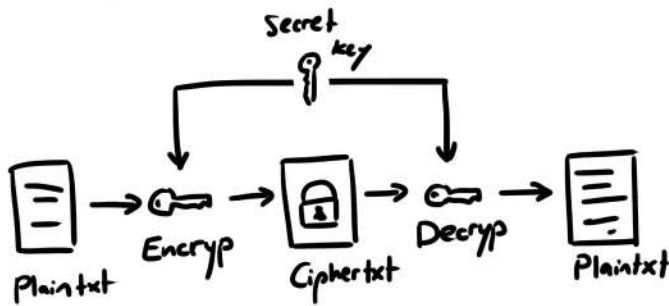


# Chapter 2

## Symmetric Encryption



- disadv: **key exchange**
- adv: very fast, implementable in HW

## Block Cipher

advanced encryption standard  
AES

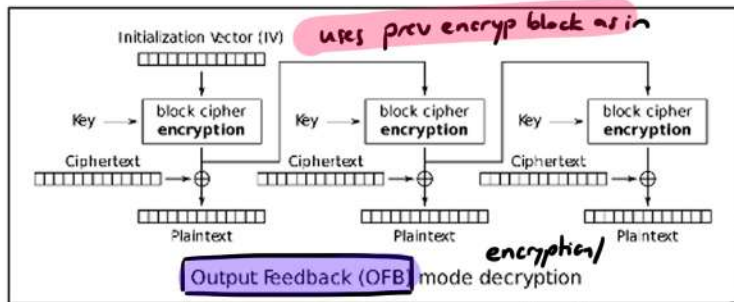
- process plain and ciphertext **in blocks** (e.g. 128bits)

## Padding

ISO Padding

- fill up **incomplete block** with "regular" pattern

## Block Cipher Operation Modes



## Electronic Code Book (ECB)

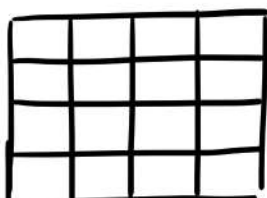
← better than

- txt block fixed size + key = block of same size encrypted
- uses same key for all
- can encryp in parallel

## AES Algorithm

Round of Operations:

1. **Sub Bytes** (substitute bytes via lookup table)
2. **Shift Rows** (permute, particular num of times)
3. **Mix Columns** (matrix mult, each col mult matrix)
4. **Add Round Keys** (XOR with corresponding round key)



4x4 bytes (128bits)

→ 10 Rounds

## Stream Cipher

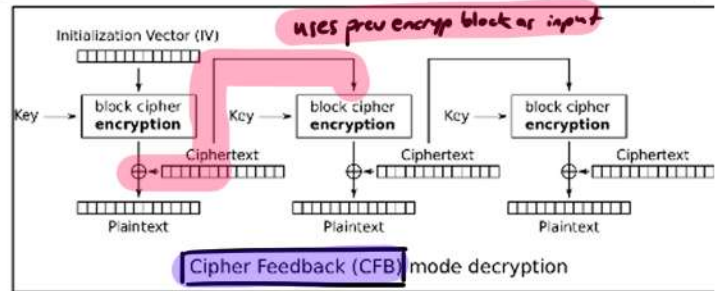
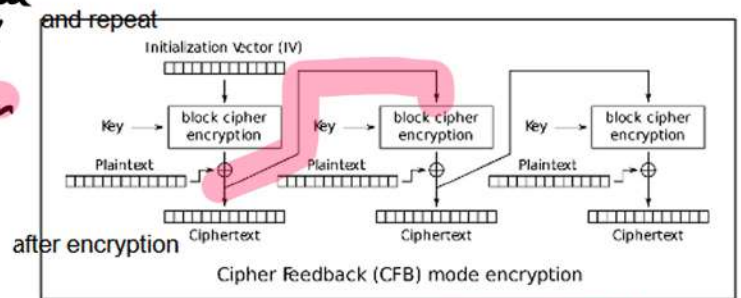
ChaCha20

- key of **fixed length**
- generate data stream of arbitrary length
- convert each bit of plain to cipher using **XOR**



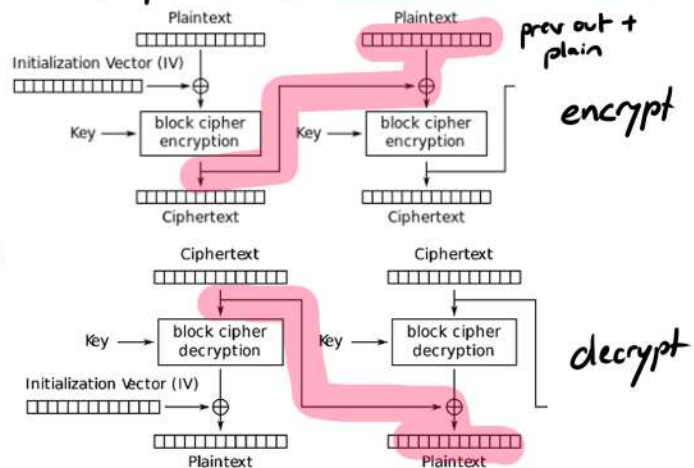
## Variants

- **Synchronous stream ciphering** → gen key stream **independently** of plain or key text
- **self-synchronizing stream ciphering** → **depends on prev encryp bit**



## Cipher Block Chaining (CBC)

- encryp of block **depends on predecessor blocks**



## Init Vector

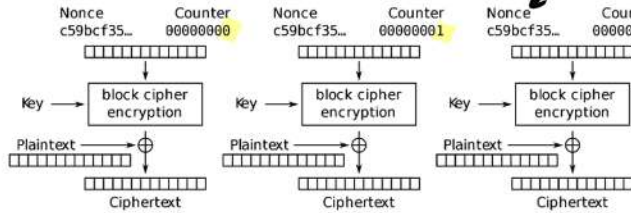
- must not be secret
- should be random and different for each msg
- same size as block (AES 128 bit)

errors:

- use key 9s IV
  - set IV to 0 or other fixed val
  - gen IV from psud
- not random and possibly exposed

## Counter Block Mode (CBM)

number used only once



Counter (CTR) mode encryption

$$C_i = P_i \oplus E_k(ctr_i)$$

## Password Based Encryption (PBE)

- gen cryptographic keys from psud
- PKCS#5 using PBKDF2 → psud based key derivation function 2
- PKCS#12 → file format that stores private keys with associated certificate in psud-protected manner

**But**: case-by-case effectiveness

- ciphertext can be modified and influences plain
- Padding Oracle Attack
- always use CBC, CTR, etc with authentication

→ Secure encryption with authentication!

## Authenticated Encryption

- authenticated if not only confidentiality but also integrity of data to be encrypted is given

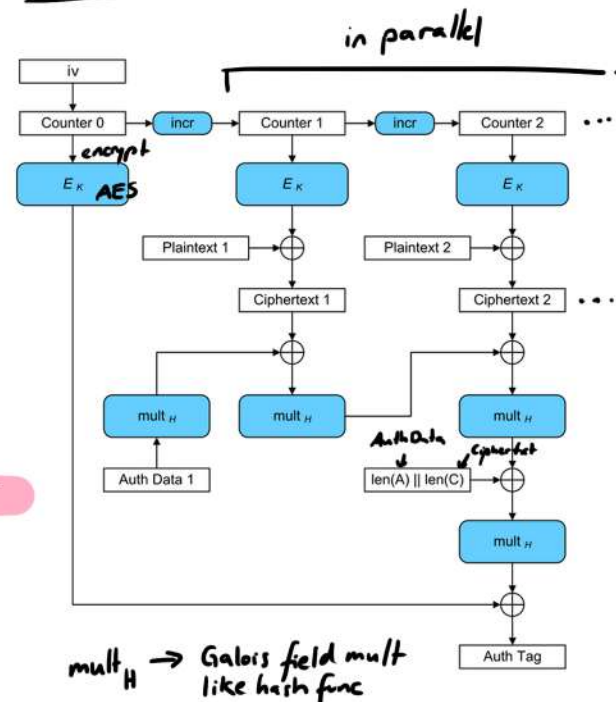
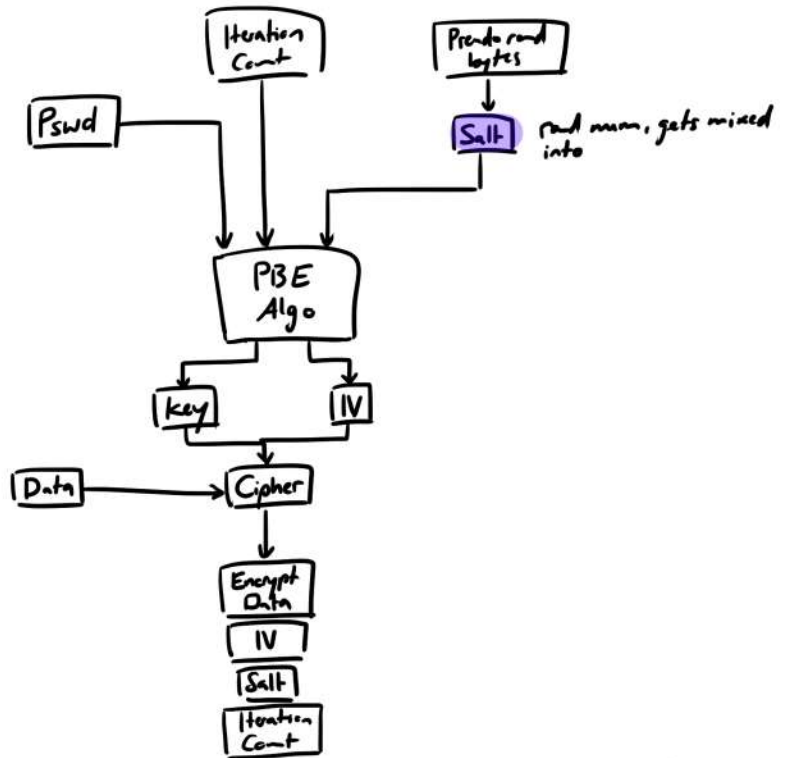
## Galois Counter Mode

AES with Counter mode for encrypt and Galois mode for authentication

→ Counter Mode: transforms block into stream cipher  
 fixed size continuous bit by bit  
 by encrypting successive values of a counter  
 → Counter + secKey = keystream  
 → keystream  $\oplus$  plaintext = ciphertext

→ Galois Mode: msg auth by gen a tag that verifies authenticity of data

↓  
 output is ciphertext and authentication tag  
 parallel processing into ciphertext possible



$mult_H \rightarrow$  Galois field mult like hash func



## Authenticated Encryption With Associated Data

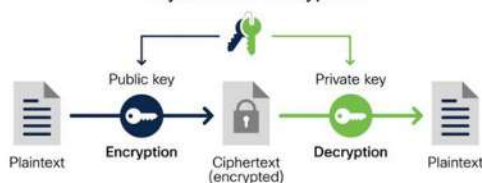
## AEAD

- provides encryption and authentication + associate additional data to encrypt msg
- associated data prevents replay attacks by providing authenticity to ciphertext  
↳ makes modifications detectable
- associated data can be e.g. addresses, ports, sequence nums, etc.
- e.g. address and seqNum as AD, it now won't work for different address and seqN

## Asymmetric Encryption

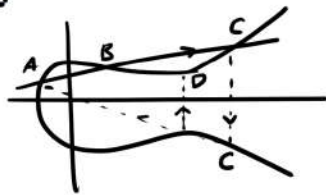
- private and public key
- disadv.: high computing effort
- adv.: key exchange no prob

Asymmetric encryption



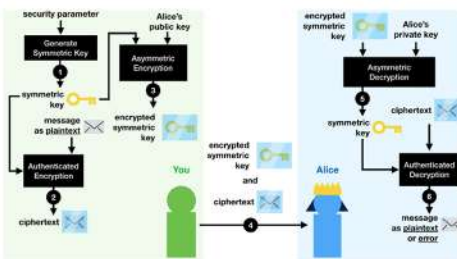
## Elliptic Curve Cryptography

- several passes with straight line through two points and x-axis reflection of intersection
- PubKey: start and endpoint
- PrivKey: num rounds



## Hybrid Encryption

- sym and asym encrypt
- msg encrypted sym
- sym key encrypted asym
- adv.: high speed encrypt
- secure key exchange



## 2.5 Practical Aspects

- ✗ Security by Obscurity → i.e. parts of encryption process must be kept secret to make it secure
- Open Design → sec does not depend on secrecy of its design or implementation
- always use crypto libraries, never own

## Compression - Encryption - Error Detection

- ↳ makes encrypt faster and cryptanalysis harder
- exploit of redundancy in txt
- ↳ add. signature/checksum → manipulations recognisable

## Key lengths

- sym:  $\geq 256$ , attk via brute force
- asym:  $\geq 3000$  (RSA), 256 (ECC), math methods

## Base 64 Encoding

- for transmission of binary data in seq of 8-bit bytes across channels that only reliably support text content
- 64 printable ASCII chars
- Principle: split 24 bits into 4x6 bits  
extend each 6 bits → 8 bits
- disadv.: msg 33% longer

## Rivest-Shamir-Adleman (RSA)

- based on difficulty of factoring large nums to hard against attack
- currently: key length of 3000
- much slower than AES

algo:

1. two large prime num  $> 2^{512}$
2. mult primes  $p, q$
3. calc Euler's Totient Function

$$\phi(n) = (p-1)(q-1)$$

4. pub exponent  $e \rightarrow 1 < e < \phi(n)$

5. calc priv exponent  $d$   
 $d \times e = 1 \pmod{\phi(n)}$   
 → solve for  $d$   
 greatest common divisor  $\text{gcd}(e, \phi(n)) = 1$   
 commonly chosen as small prime (e.g. 65537)  
 not divide  $\phi(n)$

→ PubKey (n, e)  
PrivKey (n, d)

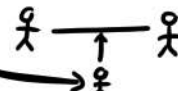
encrypt:  $c = m^e \pmod{n}$   
decrypt:  $m = c^d \pmod{n}$

## Formats for string encrypt

- XML encrypt standard
- PKCS#7 enveloped data

## Attacks

- Cryptanalysis: mathematical, statistical or implementation weaknesses
- Brute Force
- Social Engineering
- obtain keys via vulnerabilities in sys
- Side Channel Attack via phys measurements of sys (e.g. energy consumption, time consump)
- Fault Attack: causes faulty exec of crypto operation
- Man in the Middle Attack





## Key Management

- generation, storage of keys
- Withdrawal of invalid or compromised keys
- sec transmission/exchange of keys
- backup copies
- destruction of keys

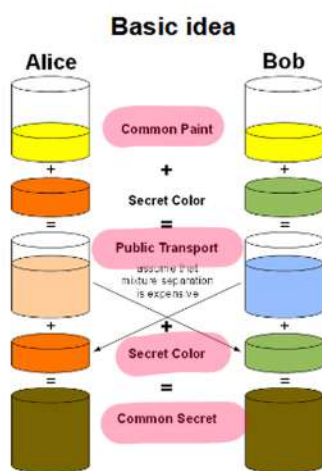
## Key Generation

- good rand nums req
- gen in secure place
- key compromised → lock-exchange-encrypt
  - key
  - all data
- secure transmission/exchange of keys, mostly based on asym cryptography

## Exchange

### Diffie-Hellman

- pub key algo for dist of keys
- not suitable for encrypt
- determining final secret even with col from pub trans is computationally expensive and impossible to compute in practical time



## Storage

- in config files, environ vars with strict access control or Password Based Encryption
- external keystores: Vaults/Secret Manager
  - ↳ access via API with authn, authz and audit logs
- chip card with pin
- key splitting (n:m principle)
  - ↑ secrets
  - ↑ secrets needed to reconstruct key
  - $n > m$

## Elliptic Curve Diffie Hellman

- EC sys params are pub

p prime number  
a,b curve parameter  
P base point on curve  
q order of P  
i cofactor

Alice:  $x \in [1, \dots, q-1]$ , sends  $Q_A = xP$

Bob: sends same with priv key y

→ same as DH, both calc privKey  $\cdot Q_{A/B}$  for common secret

## Save / Destroy

- backup copies necessary where to store and who can access it?
- exceptional situation
- key Recovery (in case of key loss)
- Key Erase (for law enforcement)
- Master Key

## Destruct:

- overwrite files and hard disk non areas
- delete cache and swap areas
- possibly destroy hardware

elliptical curve discrete logarithm problem (ECDLP)

→ computat. infeasible to find k where  $Q = k \cdot P$   
↑ target      ↑ base point